

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Deposit Contract	Documentation quality	High	
Timeline	2026-04-08 through 2026-04-10	Test quality	High	
Language	Solidity	Total Findings	4	Fixed: 3 Acknowledged: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Specification	None	Medium severity findings ⓘ	2	Fixed: 1 Acknowledged: 1
Source Code	- Polymarket/equities-monorepo ↗ #7dd0575 ↗	Low severity findings ⓘ	1	Fixed: 1
Auditors	- Paul Clemson Auditing Engineer - Zeeshan Meghji Auditing Engineer - Tim Sigl Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	1	Fixed: 1

Summary of Findings

This audit focused on the smart contracts of the Polymarket Equities Monorepo. The protocol's smart contracts allow users to deposit supported ERC-20 tokens, which can then be used to trade off-chain.

User deposit balances are not stored on chain, but instead the protocol periodically submits updated Merkle roots with leaves representing `(account, token, balance)`. Users can then request withdrawals either off-chain, by signing a request that the operator submits directly, or on-chain by calling the `requestWithdrawal()` function. The operator is then responsible for ensuring these withdrawal requests are processed in a timely manner. If a queued request is not fulfilled before its timeout, the system moves to "escape-hatch" mode, which pauses normal operations and allows users to directly withdraw their funds based on the most recently provided Merkle root. There is then logic in place for the protocol to handle recovering from "escape-hatch" mode whilst still ensuring there is a sufficient grace period for users to withdraw from the protocol before normal operations resume.

The audit found no high-severity security issues in the codebase. However, two medium severity issues were noted, as well as a number of low and informational severity issues were found, plus a number of auditor suggestions were made to improve the overall quality of the codebase. Of the medium severity issues **POL-EQ-1** related to a potential scenario where a user could withdraw their funds twice if their on-chain and off-chain withdrawal requests were using a different `stateRoot`. The other medium severity issue **POL-EQ-2** related to missing fee accounting logic within the protocol, which would prevent the contract owner from safely withdrawing earned fees.

The smart contract test suite has strong coverage, and has unit tests covering the majority of potential flows across both the "standard" and "escape-hatch" modes the protocol can be in.

Fix-Review Update 2026-04-16:

During the fix review period the client fixed issues **POL-EQ-1**, **POL-EQ-3**, and **POL-EQ-4** and acknowledged the issue **POL-EQ-2** providing operational context about how this issue will be handled off-chain. The client also chose to fix or acknowledge the auditor suggestions put forward during this audit.

Additionally, the codebase's strong test coverage remains largely unchanged between the initial report and the final fix commit. During the fix review period the project also strengthened its documentation to include more context about the assumptions made of the off-chain system.

ID	DESCRIPTION	SEVERITY	STATUS
POL-EQ-1	Missing On-Chain Prevention of Cross-Root Double Withdrawals	● Medium ⓘ	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
POL-EQ-2	Missing Fee Handling Logic	• Medium ⓘ	Acknowledged
POL-EQ-3	Grace Period Can Be Reduced During Escape Hatch, Shortening User Redemption Window	• Low ⓘ	Fixed
POL-EQ-4	Unbounded Loop in <code>getPendingWithdrawals()</code> Function	• Informational ⓘ	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

*i***Disclaimer**

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/Polymarket/equities-monorepo`

Included Paths: `contracts/src/`

Operational Considerations

The following assumptions are made of the codebase:

- **Escape Hatch Withdrawal Liquidity:** `withdraw()` pays from the contract's current token balance without per user allocation. In a scenario where the contract is underfunded early users can redeem their funds whilst later users can receive less or zero. Operators

should make sure there is always enough liquidity in the contract to prevent bank run scenarios.

- `EscapeHatchActivated` **Event Lags Escape Hatch Activation:** The `escapeHatch` flag and `EscapeHatchActivated` event are only set when `_activateEscapeHatch()` runs, but `inEscapeHatch()` can already be true while a request has expired because it returns `escapeHatch || _hasExpiredRequest()`. Integrators and operators who need early detection should not rely on that event or on the boolean alone, and should use `inEscapeHatch()`.
- **Operator Controlled Withdrawal Processing:** Users must trust the operator to process withdrawal requests and not censor specific accounts by repeatedly rejecting their requests. Since timely rejections do not trigger the escape hatch, an operator that repeatedly rejects withdrawal requests can keep a user's withdrawal flow blocked indefinitely.
- **Withdraw Cooldown Prevents Multi-Token Withdrawal:** If a user has deposited two different tokens they cannot withdraw both without waiting the `withdrawCooldown` duration between requests.
- **Single Leaf Per User/Token Pair Must be Enforced Off-chain:** Each leaf in the merkle tree is a hash of `(user, token, balance)`. Nothing on-chain prevents there being multiple leaves present where the `user`, `token` values are identical but with different `balance` values. It is assumed that the off-chain systems will ensure that there is only one leaf in the tree for a given `user / token`.
- **Withdrawal Timeout Must Give Operator Sufficient Time to Process Requests:** If the `withdrawalTimeout` value is too low a malicious user may be able to force the protocol into escape hatch mode by creating more withdrawal request than can be handled by the operator before the `withdrawalTimeout` has passed.
- **Stale Withdrawals During Escape Hatch Phase:** Operators can no longer modify the state root during the escape hatch phase. If the user's off-chain balance changes through trading or other activity after the most recent `stateRoot` update, they will still be able to withdraw against their committed balance before the escape hatch phase began. In the event that the smart contract enters escape hatch mode, it is important that off-chain trading is paused immediately to protect against the outlined scenario.
- **Request Process Ordering:** If the operator processes an off-chain request while the user has an pending on-chain request the on-chain request may not be able to be fulfilled. The operator is trusted to check if user already has a pending on-chain request before fulfilling an off-chain request.
- **Off-Chain Indexing Required for Fee Transparency:** The off-chain mitigations in place to handle [POL-EQ-2](#) mean earned fee withdrawals will be indistinguishable from standard user withdrawals. This reduces the transparency of the protocol and sufficient off-chain indexing should be implemented to allow users to check the amounts of fees earned by the protocol and the total amounts that have been withdrawn.

Key Actors And Their Capabilities

The two authorized roles in the protocol are the `owner` and the `operator`.

Exchange Owner

The contract's owner is responsible for the following:

- Upgrading to a new implementation contract when necessary.
- Transferring ownership to a new `owner` when necessary.
- Adding and removing the operator role via the `setOperator()` function.
- Adding and removing supported ERC-20 assets via the `setAsset()` function.
- Setting the operator withdrawal timeout via the `setWithdrawalTimeout()` function.
- Setting the standard user withdraw cooldown via the `setWithdrawCooldowns()` function.
- Setting the grace period necessary before the protocol can recover from "escape-hatch" mode, via the `setResumeGracePeriod()` function.
- Calling the `prepareResume()` function to begin recovery from "escape-hatch" mode.
- Calling the `clearQueue()` function to clear all outstanding withdrawal request before the protocol can resume.
- Calling the `resume()` function to finalize the recovery from "escape-hatch" mode.

Exchange Operator

The operator is responsible for the following:

- Committing the latest `stateRoot` via the `commitStateRoot()` function.
- Handling direct withdrawals with user signatures via the `processWithdrawal()` function.
- Fulfilling on-chain withdrawal requests via the `fulfillWithdrawal()` function.
- Rejected on-chain withdrawal requests via the `rejectWithdrawal()` function. The operator is trusted to only reject requests that did not provide the necessary `fee` or do not have the required off-chain token balance to fulfill the request.

Findings

POL-EQ-1

Missing On-Chain Prevention of Cross-Root Double Withdrawals

• Medium ⓘ Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `66c3bd07a1957e306f7cd3a4d2ceee82a77dab86`.
The client provided the following explanation:

Rather than adding a `hasPendingWithdrawal` check to `processWithdrawal()`, the fix eliminates the root cause by preventing `commitStateRoot()` from updating the state root while any on-chain withdrawal requests are still pending (if `(pendingRequestStart < pendingRequestIds.length)` revert `PendingRequestsExist()`). This makes it impossible for two different roots to coexist with an unfulfilled request, so the cross-root double-withdrawal scenario cannot arise.

To prevent a griefing vector where users spam `requestWithdrawal` to block state root updates indefinitely, a `MAX_PENDING_REQUESTS = 50` cap is enforced on the pending queue.

An additional fix to introduce minimum deposit amounts was included in the commit `0619779429b815f6c573052e3288598d5cfff74a`.

File(s) affected: `contracts/src/Exchange.sol`

Description: The `withdrawn` mapping that prevents over-withdrawal is keyed by state root: `withdrawn[root][token][account]`. The `processWithdrawal()` function (off-chain withdrawal path) uses the current `stateRoot`, while `fulfillWithdrawal()` function (on-chain request path) uses the request's stored root. If the state root changes between request creation and fulfillment, these are two independent keys in the `withdrawn` mapping. Because `processWithdrawal()` does not check `hasPendingWithdrawal[account]`, the operator can process an off-chain withdrawal against the new root and fulfill the on-chain request against the old root, allowing the user to withdraw their balance twice.

Exploit Scenario:

1. State root is R1. User has 100 USDC balance (proven in R1). User calls `requestWithdrawal` for 100 USDC. Request stores `root = R1`.
2. The operator commits a new state root R2 (user still shows 100 USDC in R2 because the pending request hasn't been settled yet).
3. The user signs a separate off-chain withdrawal for 100 USDC.
4. The operator calls `processWithdrawal` against R2. `withdrawn[R2][USDC][user] = 100`. User receives ~100 USDC.
5. The operator calls `fulfillWithdrawal`. It reads `req.root = R1`. `withdrawn[R1][USDC][user]` is still 0, so `remaining = 100`. User receives another ~100 USDC.
6. The user extracted ~200 USDC while only having a 100 USDC balance. The excess is paid from other users' deposits.

Recommendation: Add `if (hasPendingWithdrawal[account]) revert RequestAlreadyExists();` at the start of `processWithdrawal`. This ensures the operator cannot process an off-chain withdrawal for an account that already has a pending on-chain request.

POL-EQ-2 Missing Fee Handling Logic

• Medium ⓘ Acknowledged

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

The operator will designate an address to receive fees, and this information will be stored offchain in the transition data, which will be made public.

With each trade, fees will accrue to this recipient, and the recipient's balance will be included in the state root commitment.

The fee recipient may withdraw funds through the same withdrawal mechanisms available to users, either via the API or through `requestWithdraw`.

File(s) affected: `contracts/src/Exchange.sol`

Description: In normal protocol operations, when a user's withdrawal is successful a `fee` can be deducted from their total withdrawn amount.

However, in the current implementation of the codebase there is no logic to track the amounts of `token` that the protocol has earned as fees nor is there any logic to handle the withdrawal of such fees.

Recommendation: Consider including an authorized function to allow the protocol to withdraw earned fees, as well as the necessary fee accounting logic to ensure that the protocol cannot accidentally withdraw user funds.

POL-EQ-3

Grace Period Can Be Reduced During Escape Hatch, Shortening User Redemption Window

• Low ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `96d6e8d79c4dc7d08673264d196b360da4eb9e35`.
The client provided the following explanation:

Grace period cannot be changed when escape hatch is active.

File(s) affected: contracts/src/Exchange.sol

Description: The `setResumeGracePeriod()` function has no restriction preventing it from being called during escape hatch mode. The owner can call `prepareResume()` (which records `resumePreparedAt = block.timestamp`), then immediately call `setResumeGracePeriod(1 hours)` to reduce the grace period from its current value (e.g., 7 days) to the 1-hour minimum. This effectively shortens the window that users have to review the upcoming `resume` action and withdraw before the owner sets a new state root.

Recommendation: Prevent `setResumeGracePeriod()` from being called while `resumePrepared` is true, or require that any decrease only takes effect for future `prepareResume` calls (e.g. by snapshotting the grace period at prepare time).

POL-EQ-4 Unbounded Loop in `getPendingWithdrawals()` Function

Informational ⓘ

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: 3cc66c190d58271ff6bea047b24fc006ae87d535 .
The client provided the following explanation:

Fixed as recommended.

File(s) affected: contracts/src/Exchange.sol

Description: The `getPendingWithdrawals()` function will look from `pendingRequestStart` until `pendingRequestIds.length` before returning a `WithdrawalRequest` array of all currently pending withdrawal requests.

In rare scenarios where the pending withdrawal requests array is very large the `getPendingWithdrawals()` function may fail either due to exceeding the gas cap on `eth_call` or due to the size of the `WithdrawalRequest` array becoming larger than the RPC response size limit.

Recommendation: Consider adding an alternate function `getPendingWithdrawals(uint256 n)`, which allows the next `n` requests to be fetched safely, regardless of the total amount of outstanding withdrawal requests.

Auditor Suggestions

S1 Implement a Two-Step Ownership Transfer

Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: 6a80963222ca51372fcbc97925643b01d80a148a .
The client provided the following explanation:

Fixed as recommended.

File(s) affected: contracts/src/Exchange.sol

Description: The `transferOwnership()` function immediately assigns `owner` to the provided address. A mistyped or inaccessible address permanently locks out upgrades, operator management, asset configuration, and escape hatch recovery. A two-step pattern defers the transfer until the new owner calls `acceptOwnership()`, proving control of the destination.

Recommendation: Introduce `pendingOwner`, replace direct assignment with `transferOwnership(address)` that sets `pendingOwner`, and add `acceptOwnership()` callable only by `pendingOwner` to finalize the transfer and clear `pendingOwner`. Optionally add `cancelPendingTransfer()` for the current owner.

S2 Enforce a Hard Signature Deadline

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

The upper bound of a two-sided window (timestamp + 5 minutes) will serve as the deadline.

File(s) affected: `contracts/src/Exchange.sol`

Description: `_verifyWithdrawal()` constrains `block.timestamp` relative to the signed `timestamp` via a two-sided window, not a hard upper bound. This window does not allow users to enforce an exact deadline on how long they want their signatures to be valid.

Recommendation: Replace the validity window with an explicit hard deadline: require `block.timestamp <= deadline`.

S3 Improve Documentation

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `b1f277be04fb52f00dd65791eb5e697758fbe1e0`.
The client provided the following explanation:

Fixed as recommended.

File(s) affected: `contracts/src/Exchange.sol`

Description: The following examples are areas where the protocols documentation could be improved:

- Consider documenting the assumptions made of the ERC-20 assets that will be supported by the protocol. Ideally with a list of the assets to be supported and notes on whether assets with non-standard behaviours are expected to be supported.
- The documentation above the `escapeHatch` storage variable states that it will be "automatically set when a pending withdrawal request expires without being fulfilled". However, this variable is only set when a function calling `enforceEscapeHatch()` is called, meaning the contract could be in a frozen state before the `escapeHatch` boolean has been set to true. Consider updating the documentation to clearly specify when the value is changed.
- The inline comment on the shift overflow guard in `rejectWithdrawal()` states that "shifting uint256 by ≥ 64 silently returns 0." This is inaccurate; for `uint256`, the actual silent-zero threshold is ≥ 256 . The guard at 64 is still safe because the cooldown is capped, but the comment should be corrected to avoid confusion.

Recommendation: Consider implementing the suggested documentation changes.

S4 Remove EIP-712 Signature Verification From `requestWithdrawal()`

Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `a28f4579c3e4608ffee8d01a0506351f9d6bbf89`.
The client provided the following explanation:

Fixed as recommended.

File(s) affected: `contracts/src/Exchange.sol`

Description: The `_verifyWithdrawal()` function verifies that the `account` provided has signed the `digest`, which includes all the relevant withdrawal information. However, in the case of the `requestWithdrawal()` function the `account` is always `msg.sender`, who is already implicitly agreeing to the provided withdrawal information by signing the transaction. This increases overall gas cost and forces the caller to pre-sign a message before submitting the transaction. If a unique digest is still needed for replay protection and as the withdrawal request identifier, the digest can still be computed as a simple hash of the withdrawal parameters (account, token, amount, fee, to, salt) without performing signature validations.

Recommendation: Remove the `signature` and `timestamp` parameters from `requestWithdrawal()` and replace the `_verifyWithdrawal()` call with a direct hash of the withdrawal parameters to produce the digest. The `_verifyWithdrawal()` function and the EIP-712 signature requirement should be preserved for `processWithdrawal()`, where the operator calls on behalf of the user and the signature is the only proof of user intent.



Update

Marked as "Fixed" by the client.
Addressed in: 94ca6e2d28d077742b3bf609f81a290fe51d30e5 .
The client provided the following explanation:

Fixed as recommended.

File(s) affected: contracts/src/Exchange.sol

Description: The following changes would improve the overall quality of the codebase:

- withdraw():** Consider revert if available == 0 prior to the amount > 0 check. The second amount == 0 check could then be removed.
- getPendingWithdrawals():** In scenarios where len == 0 consider reverting or returning early.

Recommendation: Consider implementing the suggested improvements.

Definitions

- High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- Undetermined** – The impact of the issue is uncertain.
- Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: https://github.com/Polymarket/equities-monorepo

- 460...1ae ./contracts/src/Exchange.sol
- 6a5...d6f ./contracts/src/ExchangeStorage.sol
- 5d9...756 ./contracts/src/IXchange.sol

Test Suite Results

The project's tests can be run with the following commands:

```
forge build
forge test
```

```
Ran 113 tests for test/Exchange.t.sol:ExchangeTest
[PASS] test_clearQueue_clearsInBatches() (gas: 458479)
[PASS] test_clearQueue_revertsIfNotOwner() (gas: 23351)
[PASS] test_clearQueue_revertsIfNotPrepared() (gas: 26101)
[PASS] test_commitStateRoot() (gas: 57274)
[PASS] test_commitStateRoot_revertsIfNotOperator() (gas: 20853)
[PASS] test_commitStateRoot_revertsInEscapeHatch() (gas: 448962)
[PASS] test_commitStateRoot_revertsOnZeroRoot() (gas: 28010)
[PASS] test_deposit() (gas: 81705)
[PASS] test_depositWithPermit() (gas: 128727)
[PASS] test_depositWithPermit_noPreApproval() (gas: 130037)
[PASS] test_depositWithPermit_revertsInEscapeHatch() (gas: 462285)
[PASS] test_depositWithPermit_revertsOnZeroAddress() (gas: 83692)
[PASS] test_depositWithPermit_revertsOnZeroAmount() (gas: 82607)
[PASS] test_deposit_emitsEvent() (gas: 80989)
[PASS] test_deposit_revertsInEscapeHatch() (gas: 451093)
[PASS] test_deposit_revertsOnUnsupportedToken() (gas: 1622093)
[PASS] test_deposit_revertsOnZeroAddress() (gas: 31062)
[PASS] test_deposit_revertsOnZeroAmount() (gas: 31655)
[PASS] test_escapeHatch_autoActivatesOnExpiredRequest() (gas: 449800)
[PASS] test_escapeHatch_blocksDeposit() (gas: 452128)
[PASS] test_escapeHatch_blocksOperatorActions() (gas: 461796)
[PASS] test_escapeHatch_blocksProcessWithdrawal() (gas: 471490)
[PASS] test_escapeHatch_blocksRequestWithdrawal() (gas: 470210)
[PASS] test_escapeHatch_notActiveIfFulfilledBeforeDeadline() (gas: 458083)
[PASS] test_fulfillWithdrawal() (gas: 463508)
[PASS] test_fulfillWithdrawal_accountsAgainstRequestRoot() (gas: 473693)
[PASS] test_fulfillWithdrawal_resetsCooldown() (gas: 711283)
[PASS] test_fulfillWithdrawal_revertsIfAlreadyFulfilled() (gas: 461778)
[PASS] test_fulfillWithdrawal_revertsIfNotOperator() (gas: 449551)
[PASS] test_fulfillWithdrawal_revertsIfQueuedAmountNoLongerAvailable() (gas: 596686)
[PASS] test_fulfillWithdrawal_revertsOnCorruptedZeroRoot() (gas: 478900)
[PASS] test_fulfillWithdrawal_withFee() (gas: 454863)
[PASS] test_getPendingWithdrawals_empty() (gas: 17930)
[PASS] test_inEscapeHatch_returnsFalseInitially() (gas: 20167)
[PASS] test_initialize_revertsOnDoubleInit() (gas: 23377)
[PASS] test_initialize_revertsOnZeroAddress() (gas: 5121851)
[PASS] test_mockERC20_decimals() (gas: 8959)
[PASS] test_prepareResume_resetsOnReactivation() (gas: 512828)
[PASS] test_prepareResume_revertsIfNotEscapeHatch() (gas: 28124)
[PASS] test_prepareResume_revertsIfNotOwner() (gas: 448751)
[PASS] test_prepareResume_setsFlagAndKeepsQueue() (gas: 521155)
[PASS] test_processWithdrawal() (gas: 247694)
[PASS] test_processWithdrawal_revertsIfNotOperator() (gas: 142734)
[PASS] test_processWithdrawal_revertsOnExpiredSignature() (gas: 152323)
[PASS] test_processWithdrawal_revertsOnFeeTooHigh() (gas: 59356)
[PASS] test_processWithdrawal_revertsOnFutureTimestamp() (gas: 149763)
[PASS] test_processWithdrawal_revertsOnInsufficientBalanceAfterPartial() (gas: 272618)
[PASS] test_processWithdrawal_revertsOnInvalidSignature() (gas: 162904)
[PASS] test_processWithdrawal_revertsOnNoStateRoot() (gas: 60360)
[PASS] test_processWithdrawal_revertsOnReplayedSalt() (gas: 263060)
[PASS] test_processWithdrawal_revertsOnZeroAddressTo() (gas: 60321)
[PASS] test_processWithdrawal_revertsOnZeroAmount() (gas: 58679)
[PASS] test_processWithdrawal_withFee() (gas: 248057)
[PASS] test_rejectWithdrawal() (gas: 453907)
[PASS] test_rejectWithdrawal_cooldown24h() (gas: 772050)
[PASS] test_rejectWithdrawal_cooldownCapsAtMax() (gas: 431731)
[PASS] test_rejectWithdrawal_cooldownCapsOnHighCount() (gas: 426976)
[PASS] test_rejectWithdrawal_exponentialBackoff() (gas: 1272740)
[PASS] test_rejectWithdrawal_revertsIfAlreadyFulfilled() (gas: 459169)
[PASS] test_rejectWithdrawal_revertsIfNotOperator() (gas: 448678)
[PASS] test_requestWithdrawal() (gas: 456820)
[PASS] test_requestWithdrawal_revertsIfAlreadyPending() (gas: 470202)
[PASS] test_requestWithdrawal_revertsIfNoStateRoot() (gas: 108238)
[PASS] test_requestWithdrawal_revertsInEscapeHatch() (gas: 467657)
[PASS] test_requestWithdrawal_revertsOnFeeTooHigh() (gas: 55183)
[PASS] test_requestWithdrawal_revertsOnInsufficientBalance() (gas: 155474)
[PASS] test_requestWithdrawal_revertsOnInvalidProof() (gas: 152326)
[PASS] test_requestWithdrawal_revertsOnZeroAddressTo() (gas: 52988)
[PASS] test_requestWithdrawal_revertsOnZeroAmount() (gas: 52956)
```



```
[PASS] test_requestWithdrawal_withFee() (gas: 450467)
[PASS] test_resume_recoversAfterPrepare() (gas: 447576)
[PASS] test_resume_revertsDuringGracePeriod() (gas: 458020)
[PASS] test_resume_revertsIfNotOwner() (gas: 22964)
[PASS] test_resume_revertsIfNotPrepared() (gas: 478584)
[PASS] test_resume_revertsIfQueueNotCleared() (gas: 509434)
[PASS] test_resume_revertsOnZeroStateRoot() (gas: 460783)
[PASS] test_setAsset() (gas: 1634947)
[PASS] test_setAsset_revertsIfNotOwner() (gas: 24290)
[PASS] test_setOperator() (gas: 48072)
[PASS] test_setOperator_revertsIfNotOwner() (gas: 25562)
[PASS] test_setResumeGracePeriod() (gas: 34645)
[PASS] test_setResumeGracePeriod_revertsIfAboveMax() (gas: 23621)
[PASS] test_setResumeGracePeriod_revertsIfBelowMin() (gas: 22611)
[PASS] test_setResumeGracePeriod_revertsIfNotOwner() (gas: 21626)
[PASS] test_setWithdrawCooldowns() (gas: 42766)
[PASS] test_setWithdrawCooldowns_revertsIfBaseAboveMax() (gas: 23219)
[PASS] test_setWithdrawCooldowns_revertsIfBaseBelowMin() (gas: 22968)
[PASS] test_setWithdrawCooldowns_revertsIfBaseGtMax() (gas: 23659)
[PASS] test_setWithdrawCooldowns_revertsIfMaxAboveMax() (gas: 22387)
[PASS] test_setWithdrawCooldowns_revertsIfMaxBelowMin() (gas: 21952)
[PASS] test_setWithdrawCooldowns_revertsIfNotOwner() (gas: 22463)
[PASS] test_setWithdrawalTimeout() (gas: 38773)
[PASS] test_setWithdrawalTimeout_allowsIncreasingWithPendingRequests() (gas: 456892)
[PASS] test_setWithdrawalTimeout_revertsIfAboveMax() (gas: 21965)
[PASS] test_setWithdrawalTimeout_revertsIfBelowMin() (gas: 21898)
[PASS] test_setWithdrawalTimeout_revertsIfNotOwner() (gas: 23328)
[PASS] test_setWithdrawalTimeout_revertsIfShorteningWithPendingRequests() (gas: 450404)
[PASS] test_transferOwnership() (gas: 33719)
[PASS] test_transferOwnership_revertsIfNotOwner() (gas: 24593)
[PASS] test_transferOwnership_revertsOnZeroAddress() (gas: 23906)
[PASS] test_upgrade_authorizedByOwner() (gas: 5060777)
[PASS] test_upgrade_revertsIfNotOwner() (gas: 5054509)
[PASS] test_withdraw_autoActivates() (gas: 492955)
[PASS] test_withdraw_capsToAvailableBalance() (gas: 510050)
[PASS] test_withdraw_negativeBalanceReverts() (gas: 507189)
[PASS] test_withdraw_revertsIfAlreadyClaimed() (gas: 502138)
[PASS] test_withdraw_revertsIfNoExpiredRequests() (gas: 120015)
[PASS] test_withdraw_revertsIfNoStateRoot() (gas: 25294)
[PASS] test_withdraw_revertsOnInvalidProof() (gas: 479333)
[PASS] test_withdraw_revertsOnZeroAvailableBalance() (gas: 476402)
[PASS] test_withdraw_revertsOnZeroBalance() (gas: 480580)
[PASS] test_withdraw_twoUsers() (gas: 566476)
[PASS] test_withdraw_withMerkleProof() (gas: 500855)
Suite result: ok. 113 passed; 0 failed; 0 skipped; finished in 10.06ms (112.68ms CPU time)

Ran 1 test suite in 97.01ms (10.06ms CPU time): 113 tests passed, 0 failed, 0 skipped (113 total tests)
```

Code Coverage

The code coverage data can be obtained by running the following commands:

```
forge coverage --ir-minimum --no-match-coverage "script"
```

File	% Lines	% Statements	% Branches	% Funcs
src/Exchange.sol	95.48% (211/221)	96.25% (257/267)	98.11% (52/53)	100.00% (37/37)
test/mocks/MockERC20.sol	100.00% (10/10)	100.00% (6/6)	100.00% (0/0)	100.00% (5/5)
Total	95.67% (221/231)	96.34% (263/273)	98.11% (52/53)	100.00% (42/42)

Changelog

- 2026-04-10 - Initial report
- 2026-04-17 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

